MACHINE LEARNING IN PHYSICS
CONVOLUTIONAL NEURAL NETWORKS

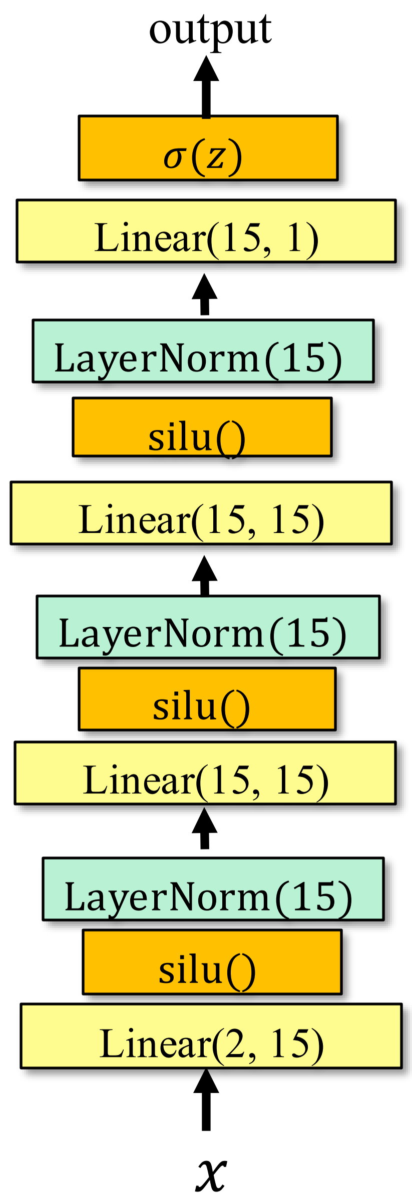
HARRISON B. PROSPER

PHY6937 / PHY4936

Recap

We used the simplest machine learning architecture, namely a multi-layer perceptron (MLP) or feed-forward neural network (FFNN), such as the one shown here, to review a few key concepts in ML, including

1. the loss function,
2. the empirical risk (or average loss),
3. the risk functional and the equation $\int dy \, \frac{\partial L}{\partial f} p(y \mid x) = 0$,
4. and empirical risk minimization via stochastic gradient descent.



The plan for the next several lectures is to introduce the following machine learning models:

1. Convolutional neural networks (CNN)
2. Physics-informed neural networks (PINN)
3. Graph neural networks (GNN)
4. Autoencoders (AE)
5. Boosted decision trees (BDT)
6. Flow and diffusion models
7. Transformer neural networks (TNN)

Outline

- Introduction
- Multi-Class Classification
- Convolutional Neural Networks
- A Galaxy Classifier
- Summary

Introduction (1)

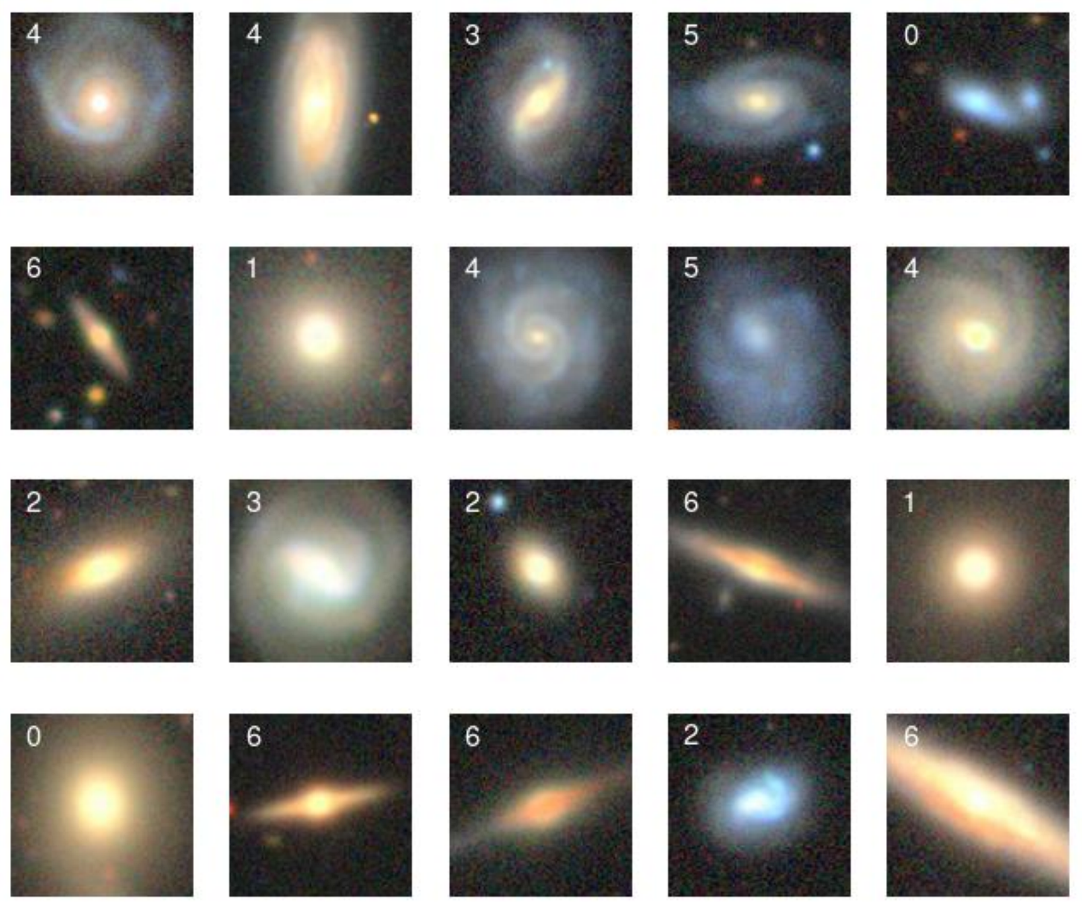
- A great deal of our behavior is guided by our ability to interpret visual data, an ability that is now available in handheld devices.

- The breakthrough that has made this possible is the **convolutional neural network** (CNN) and its many, many, variations.

- Today, we'll discuss CNNs using the classification of galaxies as an example.

Introduction (2)

We shall use the images from the Galaxy 10 DECals Dataset at the astroNN website.



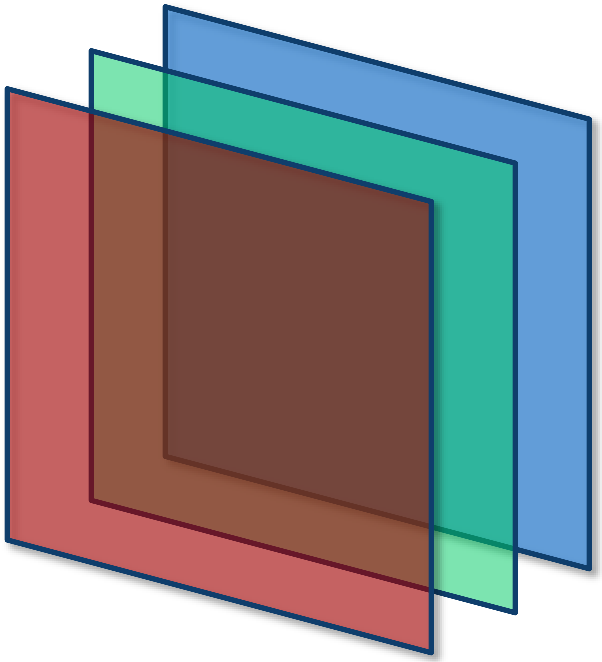
Goal: classify galaxies using images like these.

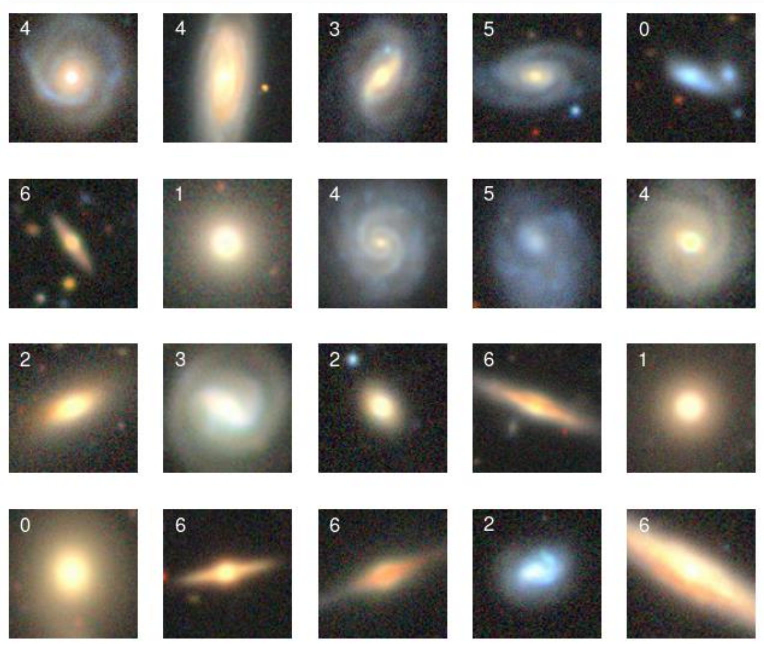
https://astronn.readthedocs.io/en/stable/galaxy10.html

Introduction (3)

Data set

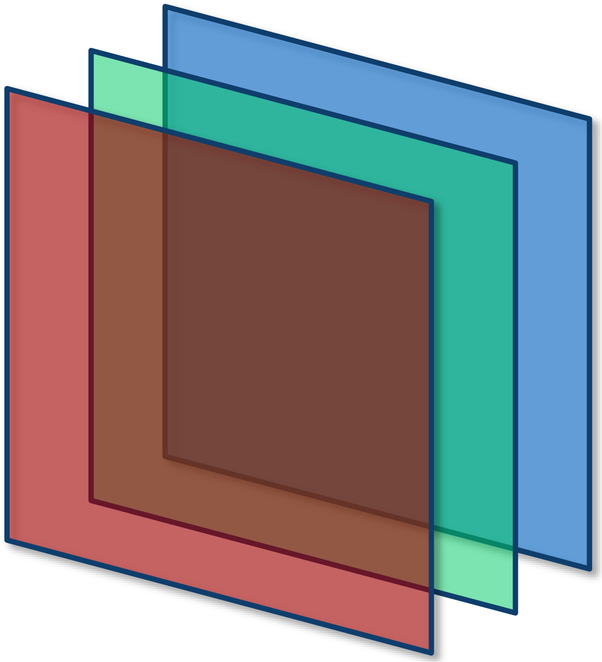
1. The data set comprises 12,600 images divided into 7 galaxy morphology classes with 1800 galaxies per class.
2. Each 3-channel image is cropped to 96 x 96 pixels.

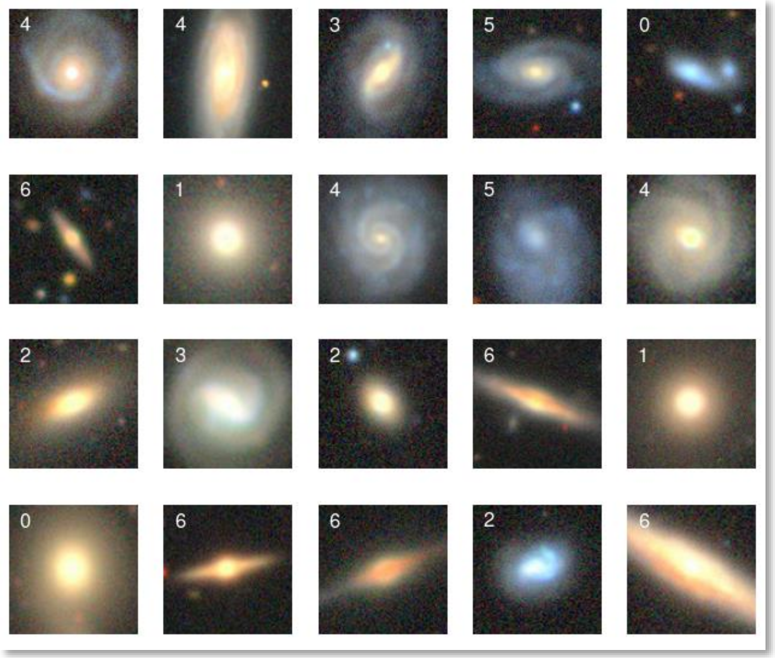




Introduction (4)

This a **multi-class** problem, so we need a generalization of binary classification.





MULTI-CLASS CLASSIFICATION

Multi-Class Classification (1)

In the galaxy classification problem, we have $K = 7$ classes. Therefore, we need K functions, $f_k(x)$, which satisfy

$$\sum_{k=0}^{K-1} f_k(x) = 1, \qquad \forall x$$

that approximate each of the K class probabilities,

$$p(k \mid x) = \frac{p(x \mid k)\, \pi(k)}{p(x)}$$

Multi-Class Classification (2)

Recall that the risk functional can be written as

$$R[f] = \mathbb{E}_{x\sim p(x)}[\boldsymbol{\mathcal{L}}(\boldsymbol{x}, \boldsymbol{f})]$$

with

$$\boldsymbol{\mathcal{L}}(\boldsymbol{x}, \boldsymbol{f}) = \int dy\, \boldsymbol{L}(\boldsymbol{y}, \boldsymbol{f})\, p(y|\, x)$$

However, the values of y are discrete: $y = 0, \cdots, K-1$.
Therefore,

$$\boldsymbol{\mathcal{L}}(\boldsymbol{x}, \boldsymbol{f}) = \sum_{k=1}^{K-1} L_k\, p_k$$

where $L_k \equiv L(k, f_k)$ and $p_k \equiv p(k\,|x)$.

Multi-Class Classification (3)

A perfect model, f_k, would satisfy $f_k = p_k$.

The question is: What loss function yields this result?

We saw in the previous lecture that the optimal solution $f_k = p_k$ is found by minimizing the risk functional $R[f]$.

But we need to account for the constraint

$$\sum_{k=1}^{K-1} f_k = 1$$

An Aside (1)

An elegant way to account for a constraint in a minimization task is the method of **Lagrange multipliers**. Consider the following classic problem: Find the coefficients c_i in the sum

$$F = \sum_{i=0}^{K-1} c_i\, x_i$$

subject to the constraint

$$\sum_{i=0}^{K-1} c_i = 1$$

that minimizes the variance of the weighted sum F.

An Aside (2)

Step 1: Find a formula for the variance of

$$F = \sum_{i=0}^{K-1} c_i \, x_i \quad \Longrightarrow \quad \Delta F = \sum_{i=0}^{K-1} c_i \, \Delta x_i$$

$$\mathbb{E}[(\Delta F)^2] = \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} c_i c_j \mathbb{E}\left[\Delta x_i \Delta x_j\right]$$

Assuming the x_i are statistically independent, we find

$$\mathbb{V}[F] = \sum_{i=0}^{K-1} c_i^2 \, \mathbb{V}_i$$

where $\mathbb{V}_i = \mathbb{E}[(\Delta x_i)^2]$.

An Aside (3)

Step 2: Add the constraint to the variance $\mathbb{V}[F]$ using a Lagrange* multiplier λ:

$$\mathbb{V}[F] = \sum_{i=0}^{K-1} c_i^2\, \mathbb{V}_i + \boldsymbol{\lambda}\left(\sum_{i=0}^{K-1} \boldsymbol{c_i} - \mathbf{1}\right)$$

Step 3: Minimize $\mathbb{V}[F]$ with respect to c_j

$$\frac{\partial \mathbb{V}}{\partial c_j} = 2c_j\mathbb{V}_j + \lambda = 0 \implies c_j = -\frac{\lambda}{2\mathbb{V}_j}$$

*Joseph-Louis Lagrange (1736 – 1813)

An Aside (4)

Step 4: Impose the constraint

$$\sum_{i=0}^{K-1} c_i = 1$$

that is,

$$-\frac{\lambda}{2} \sum_{i=0}^{K-1} \frac{1}{\mathbb{V}_i} = 1$$

Therefore,

$$c_j = \frac{1}{\mathbb{V}_j} \Big/ \sum_{i=0}^{K-1} \frac{1}{\mathbb{V}_i}$$

BREAK

Multi-Class Classification (4)

Let's apply the Lagrange method to the problem at hand.

$$R[f] = \mathbb{E}_{x\sim p(x)}\left[\sum_{k=0}^{K-1} L_k\, p_k\right] + \boldsymbol{\lambda}\left(\mathbb{E}_{\boldsymbol{x\sim p(x)}}\left[\sum_{\boldsymbol{k=0}}^{\boldsymbol{K-1}} \boldsymbol{f_k} \; - \mathbf{1}\right]\right)$$

To obtain the minimum of $R[f]$ set

$$\frac{\delta R}{\delta f_j} = 0$$

This yields

$$\mathbb{E}\left[\sum_{k=0}^{K-1}\left(\frac{\partial L_k}{\partial f_j}\, p_k + \lambda \frac{\partial f_k}{\partial f_j}\right)\right] = 0$$

where, for simplicity, we've dropped the subscript $x \sim p(x)$.

Multi-Class Classification (4)

From

$$\mathbb{E}\left[\sum_{k=0}^{K-1}\left(\frac{\partial L_k}{\partial f_j}\,p_k + \lambda\frac{\partial f_k}{\partial f_j}\right)\right] = 0$$

we obtain

$$\mathbb{E}\left[\frac{\partial L_j}{\partial f_j}\,p_j + \lambda\right] = 0$$

Multi-Class Classification (5)

We want the expression

$$\mathbb{E}\left[\frac{\partial L_j}{\partial f_j}\, p_j + \lambda\right] = 0$$

to hold for every value of x and for every density $p(x)$.

This will be true *if and only if*

$$\frac{\partial L_j}{\partial f_j}\, p_j + \lambda = 0$$

Since the optimal solution is $f_j = p_j$, this implies

$$\frac{\partial L_j}{\partial f_j}\, f_j + \lambda = 0$$

Multi-Class Classification (6)

Solving

$$\frac{\partial L_j}{\partial f_j} f_j + \lambda = 0$$

for the loss, L_j, we find

$$L_j = -\lambda \log f_j$$

We can always rescale the loss so that $\lambda = 1$.

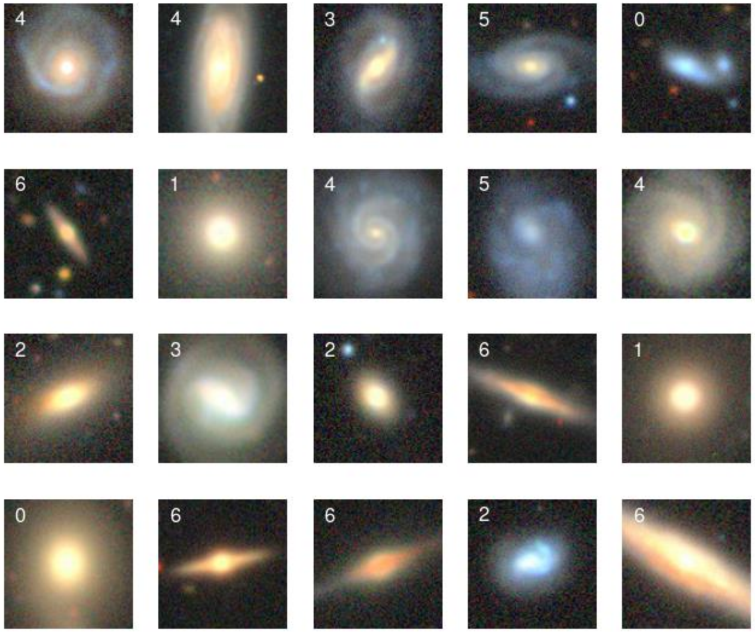
$L_j = -\log f_j$ is called the **cross-entropy loss**.

Cross Entropy Loss

The previous slide suggests that we can solve the multi-class problem by minimizing the following empirical risk

$$R(\omega) = -\frac{1}{M}\sum_{i=1}^{M} \log f_{y_i}(x_i, \omega)$$

where y_i is the class index of image i. We also assume that our model is so flexible that it can approximate $K = 7$ different probabilities.



CONVOLUTIONAL NEURAL NETWORKS (CNN)

Convolutional Neural Networks (1)

- Convolutional neural networks (CNN) are functions that extract features from 1, 2 or 3D data, which are then processed with another neural network.

- The key insight that underpins CNNs is that many data sets, for example images, contain periodicities that connect different parts of the data sets.

Harrison Prosper, Dinan, France, 2025

Convolutional Neural Networks (2)

A standard CNN comprises three types of processing layers:

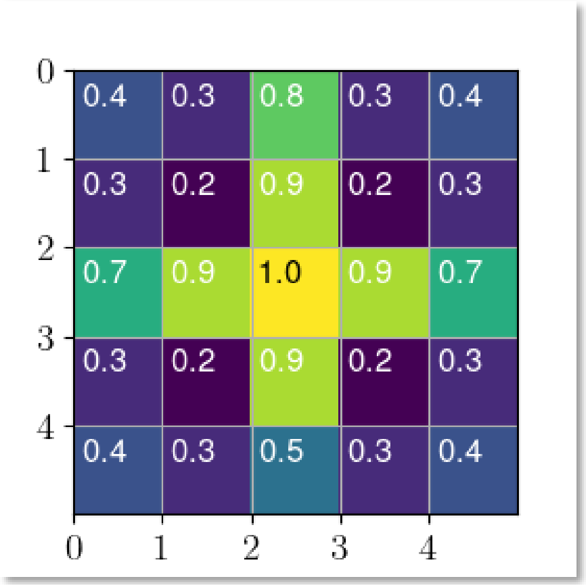
1. **Convolution**
   The data, e.g., an image, is *convolved* with one or more filters.

2. **Down-sampling**
   After applying a non-linear map to the convolved data, the latter are down-sampled.

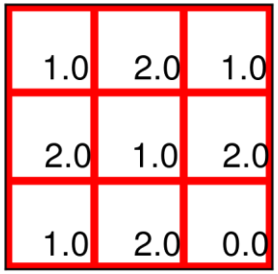
3. **Classification**
   A standard neural network classifies the convolved data.

Convolutional Neural Network

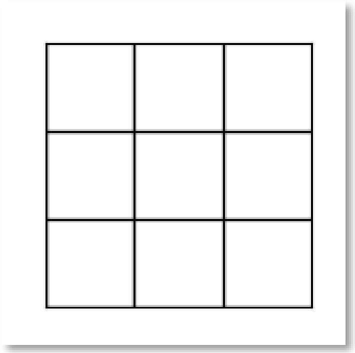
1. Convolution



(5, 5, 1)-image



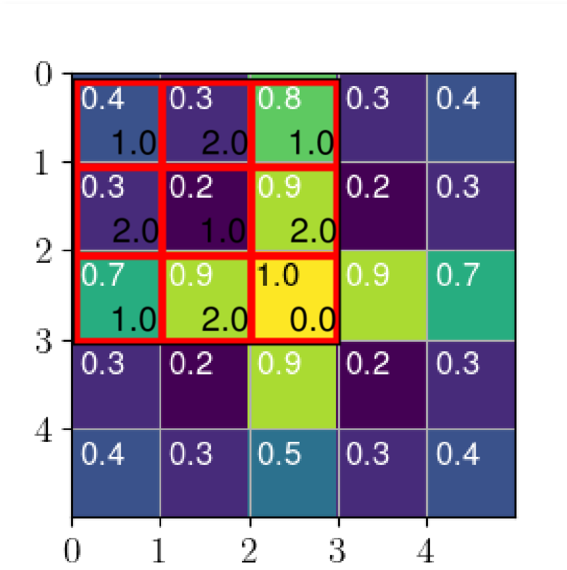
(3, 3)-filter



(3, 3, 1)-image

Convolutional Neural Network

1. Convolution



(5, 5, 1)-image

1.0	2.0	1.0
2.0	1.0	2.0
1.0	2.0	0.0

(3, 3)-filter

stride = 1

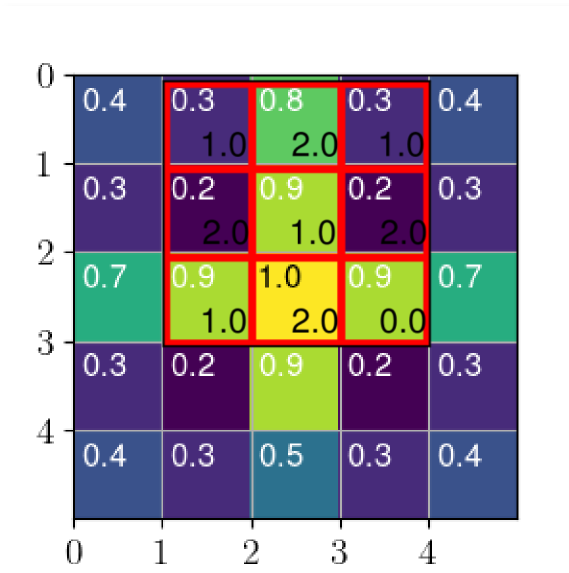
6.9		

(3, 3, 1)-image convolved

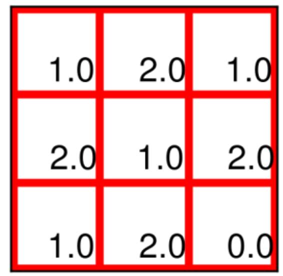
$$\mathbf{0.4} \times 1.0 + \mathbf{0.3} \times 2.0 + \mathbf{0.8} \times 1.0 + \mathbf{0.3} \times 2.0 + \mathbf{0.2} \times 1.0 + \mathbf{0.9} \times 2.0 + \mathbf{0.7} \times 1.0 + \mathbf{0.9} \times 2.0 + \mathbf{1.0} \times 0.0 = 6.9$$

Convolutional Neural Network

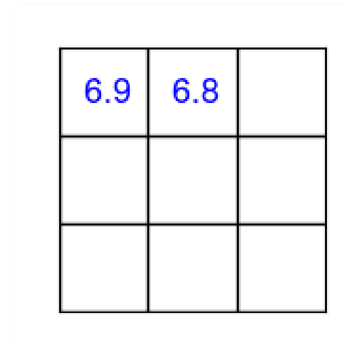
1. Convolution



(5, 5, 1)-image



(3, 3)-filter

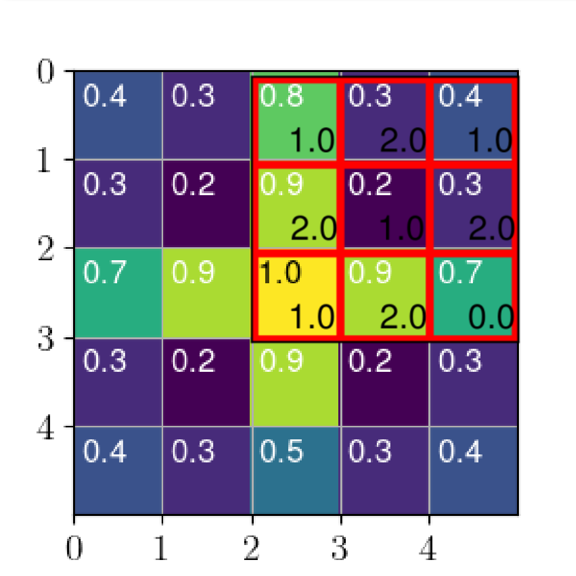


(3, 3, 1)-image convolved

$$\mathbf{0.3} \times 1.0 + \mathbf{0.8} \times 2.0 + \mathbf{0.3} \times 1.0 + \mathbf{0.2} \times 2.0 + \mathbf{0.9} \times 1.0 + \mathbf{0.2} \times 2.0 + \mathbf{0.9} \times 1.0 + \mathbf{1.0} \times 2.0 + \mathbf{0.9} \times 0.0 = 6.8$$

Convolutional Neural Network

1. Convolution



(5, 5, 1)-image

1.0	2.0	1.0
2.0	1.0	2.0
1.0	2.0	0.0

(3, 3)-filter

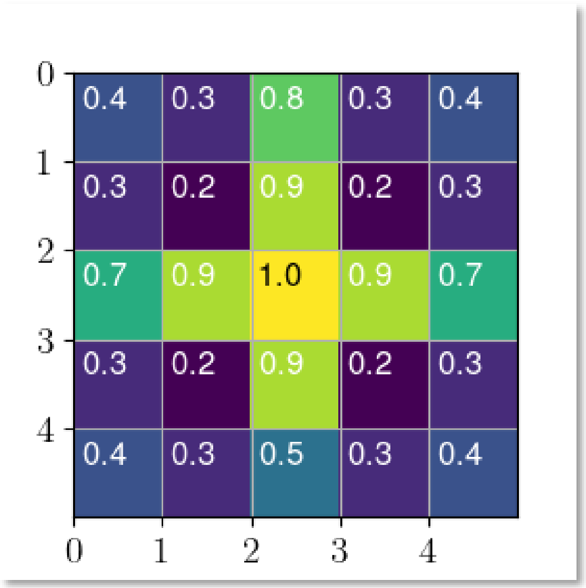
6.9	6.8	7.2

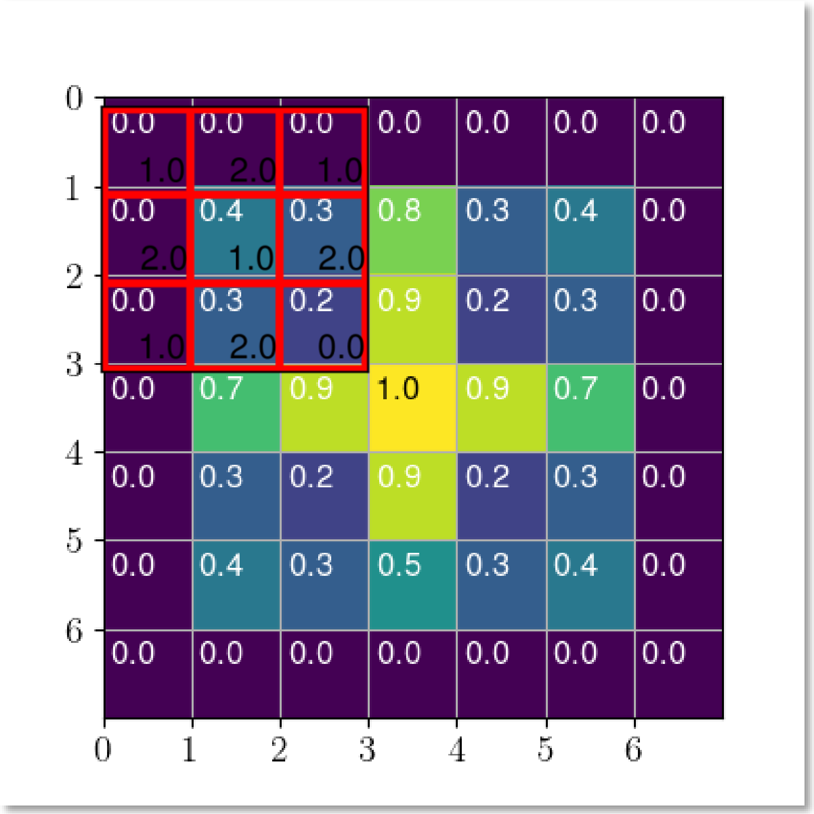
(3, 3, 1)-image (convolved)

$$\mathbf{0.8} \times 1.0 + \mathbf{0.3} \times 2.0 + \mathbf{0.4} \times 1.0 + \mathbf{0.9} \times 2.0 + \mathbf{0.2} \times 1.0 + \mathbf{0.3} \times 2.0 + \mathbf{1.0} \times 1.0 + \mathbf{0.9} \times 2.0 + \mathbf{0.7} \times 0.0 = 7.2$$

Convolutional Neural Network

Padding

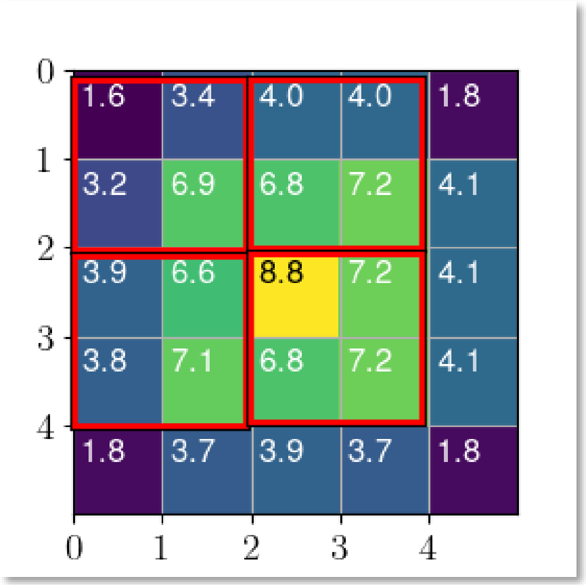




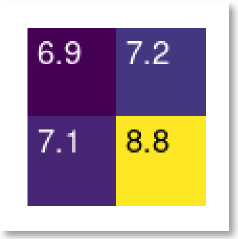
Wrapping the image with a 1–pixel border makes it possible, with a (3, 3)-filter, to maintain the image size.

Convolutional Neural Network

2. Down-sampling



Stride = 2



(2, 2, 1)-image
(down-sampled)

(5, 5, 1)-image (convolved)

MaxPool2d	Return the largest value within window.
AvgPool2d	Return the average value within window.

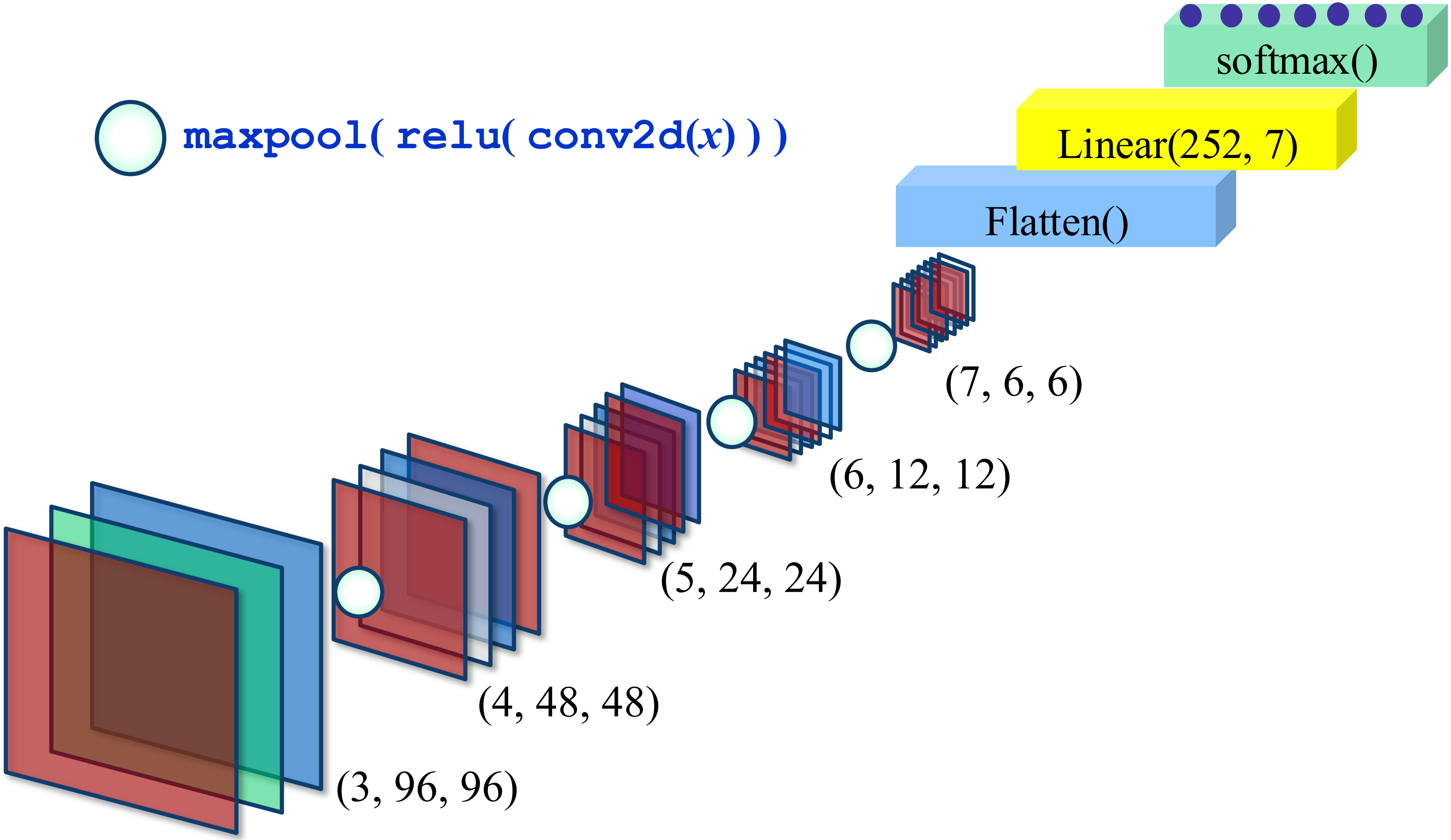
Convolutional Neural Network

3. **Classification**
   The convolved data go to a fully-connected neural network whose outputs approximate the class probabilities:

$$p(k \mid x) = \frac{p(x \mid k)\, \pi(\mathrm{k})}{p(x)}$$

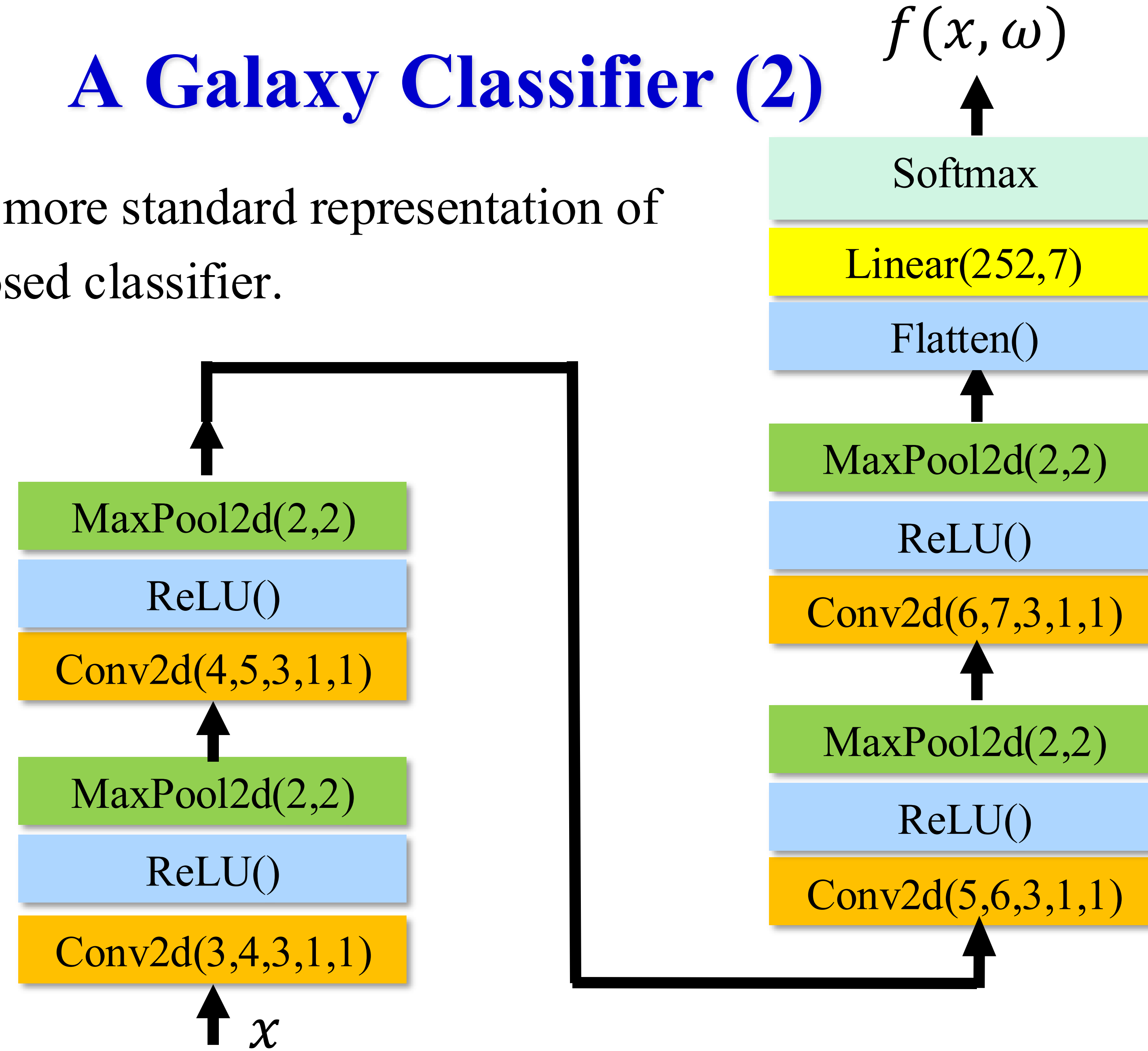
A GALAXY CLASSIFIER

A Galaxy Classifier (1)



A Galaxy Classifier (2)

Here is a more standard representation of the proposed classifier.



Summary

- Convolutional neural networks have revolutionized image classification and interpretation and are now used in many applications.

- The model exploits the fact that natural images are often locally approximately translation invariant.

- The convolutions automatically extract features from images that can later be used to interpret them.